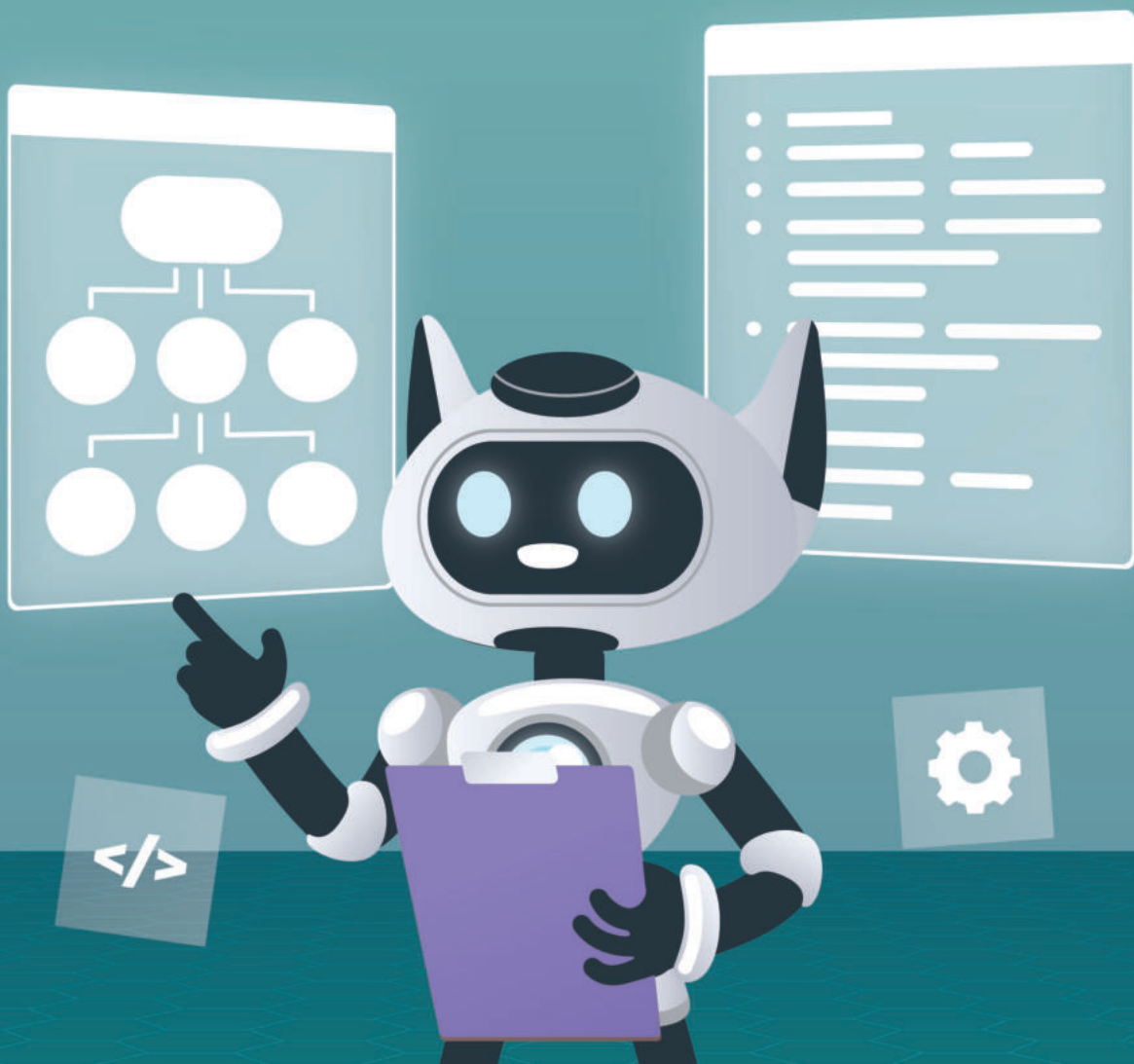


KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
REPUBLIK INDONESIA, 2024
Informatika untuk SMA/MA Kelas XI
Penulis : Dela Chaerani, dkk.
ISBN : 978-623-388-204-0 (jil.2 PDF)

Bab 3

Strategi Algoritmik, Desain Struktur Data, dan Analisis Solusi



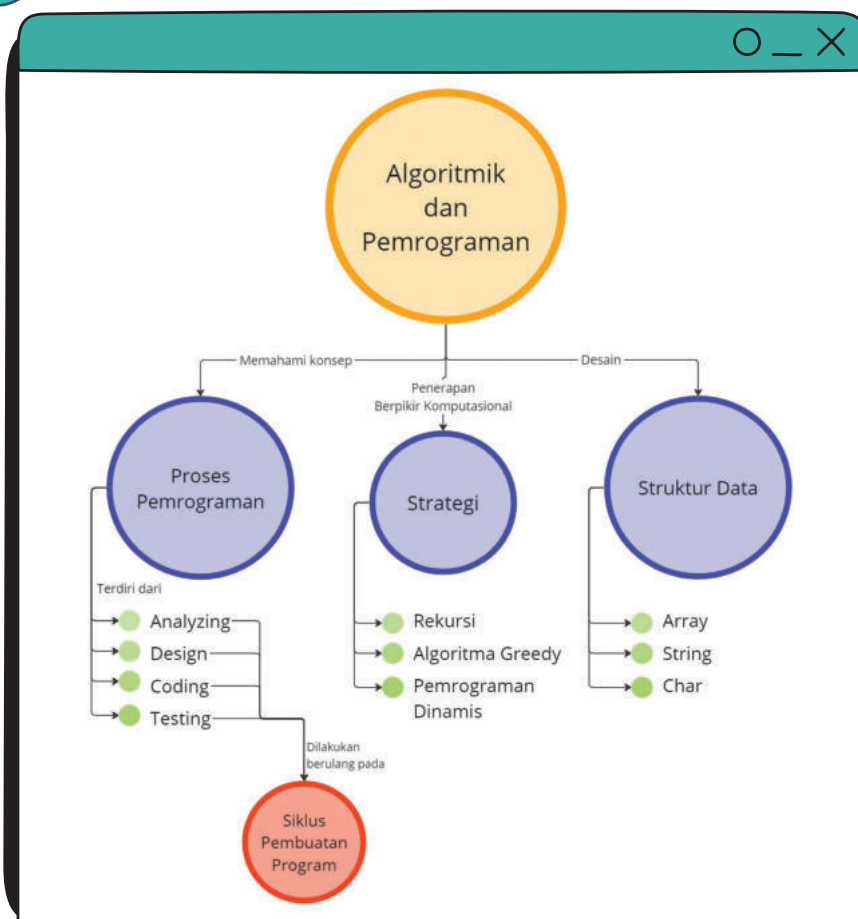


Tujuan Pembelajaran

Setelah mempelajari bab ini, kamu mampu memahami alur proses pengembangan program atau produk teknologi digital; mampu menuliskan algoritma yang efisien, efektif, dan optimal; mampu menganalisis persoalan dengan pemahamannya terhadap beberapa strategi algoritmik untuk menghasilkan beberapa alternatif solusi dari satu persoalan. Kamu juga mampu memilih dan menerapkan solusi terbaik, paling efisien, dan optimal dengan merancang struktur data yang lebih kompleks dan abstrak.



Peta Konsep

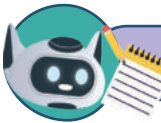


Gambar 3.1 Peta Konsep Strategi Algoritmik dan Pemrograman



Kata Kunci

- Strategi Algoritmik
- Larik (*Array*)
- Penyelesaian Masalah
- Pengkodean (*coding*)
- *Testing*



Siap-Siap Belajar

Pada fase sebelumnya, kamu sudah belajar dasar-dasar membuat program, yaitu algoritma, diagram alir, ekspresi, percabangan, pengulangan, dan fungsi serta struktur data. Akan tetapi, program yang bermanfaat tidak sekedar dapat dijalankan, tetapi bagaimana program tersebut dapat berfungsi dengan benar dan efisien sehingga dapat digunakan oleh manusia dengan nyaman. Untuk menghasilkan program demikian, diperlukan suatu kemampuan menyusun strategi algoritmik dan pemrograman.

Pernahkah kamu berpikir bagaimana program yang kita gunakan dalam kehidupan sehari-hari dikembangkan? Bagaimana para pemrogram menghasilkan program yang sangat kompleks yang kamu gunakan di kehidupan sehari-hari? Bagaimana program tersebut dirancang dan diimplementasikan dengan efisien sehingga program terasa nyaman saat digunakan?

A. Memahami Alur Proses Pengembangan Program

Pada buku Informatika SMA Kelas X, kamu telah mempelajari proses menulis kode program atau yang kita sebut sebagai *coding*. Pada kelas XI, kamu akan mempelajari kegiatan yang lebih kompleks dari *coding* yang kita sebut sebagai pemrograman. Pemrograman menurut buku *Oxford Dictionary of Computer Science* adalah seluruh aktivitas teknis yang dilakukan untuk menghasilkan suatu program, termasuk analisis kebutuhan dan seluruh langkah desain dan implementasi suatu program.

Program dapat menjadi solusi dari suatu permasalahan. Untuk menghasilkan program yang benar dan dapat membantu manusia dalam melakukan tugasnya, ada empat langkah yang dilakukan pada saat melakukan pemrograman.

1. Menganalisis Permasalahan (*Analyzing*)

Pemrogram menganalisis suatu kebutuhan atau keadaan saat ini untuk menghasilkan definisi permasalahan yang perlu diselesaikan dengan program. Permasalahan yang dianalisis bisa berupa masalah yang baru atau penyempurnaan dari solusi yang sudah ada. Kemampuan berpikir komputasional digunakan untuk mencari abstraksi dari permasalahan yang akan diselesaikan. Permasalahan yang kompleks dapat didekomposisi ke beberapa masalah yang lebih kecil, tetapi saling berkaitan. Hingga akhirnya, pemrogram akan mengenali pola permasalahan tersebut sebagai sebuah variasi dari *problem generic*. Tahap ini menghasilkan pernyataan masalah (*problem statement*) yang menjelaskan masukan (*input*), keluaran (*output*), serta batasan-batasan (*constraint*) dari program yang akan dibuat.

2. Mendesain Solusi (*Design*)

Dari pernyataan masalah ini, pemrogram merencanakan strategi untuk menghasilkan keluaran berdasarkan masukan yang diterima. Pemrogram tidak harus merencanakan solusi dari awal. Mereka dapat menggunakan solusi atau potongan solusi yang sudah ada dari permasalahan yang telah diselesaikan sebelumnya. Bahkan, jika permasalahan inti telah ditemukan, pemrogram dapat memodifikasi algoritma generik agar sesuai dengan permasalahan. Proses ini akan menghasilkan algoritma berupa narasi, *pseudocode*, atau diagram alir. Pada tahap ini, pemrogram juga akan mengevaluasi algoritma yang dibuat untuk memenuhi batasan dari permasalahan. Misalnya, apakah program dapat bekerja dengan cepat (kurang dari 1 detik) saat diberikan masukan yang berukuran besar. Menunggu membuat pengguna tidak nyaman.

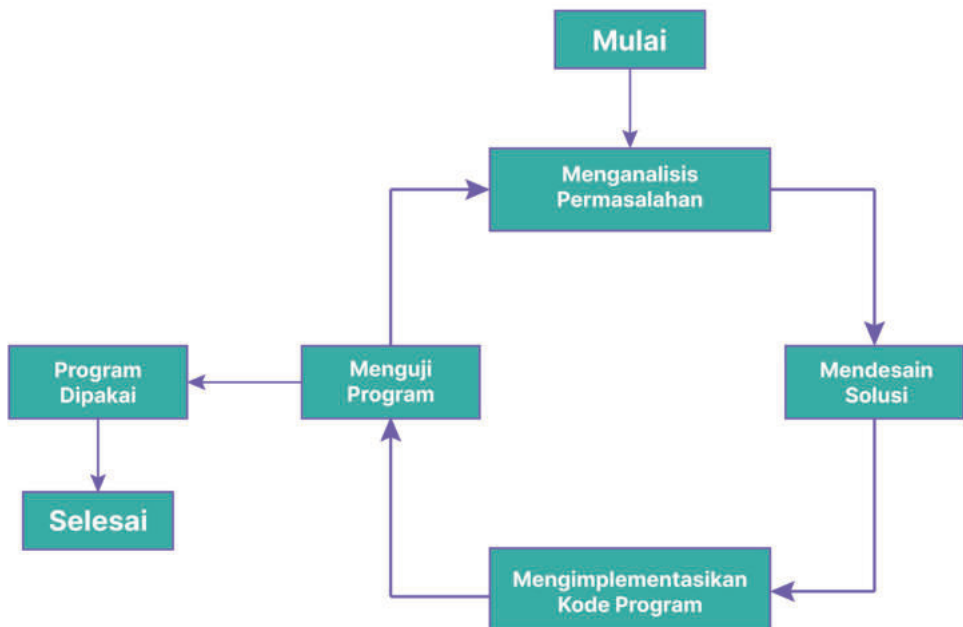
3. Mengimplementasikan Solusi dalam Bentuk Program (Coding)

Pada tahap ini, pemrogram akan menulis kode program untuk menjalankan solusi yang telah direncanakan sebelumnya dengan menggunakan suatu bahasa pemrograman. Memilih bahasa pemrograman menjadi pertimbangan di tahap ini. Selain itu, mengubah algoritma menjadi kode program juga melibatkan banyak pertimbangan teknis (misalnya, tipe data, struktur kontrol yang digunakan, dan lain-lain).

4. Menguji Program (Testing)

Setelah program dapat dijalankan, program tersebut harus diuji untuk memastikan program berjalan dengan benar sesuai dengan batasan-batasan yang diberikan. Pengujian dapat dilakukan dengan menggunakan berbagai strategi pengujian. Salah satu yang telah kamu pelajari di kelas X adalah menguji program menggunakan kasus uji (*test case*) yang dibuat sedemikian rupa sehingga mewakili seluruh kemungkinan masukan dari program.

Keempat tahap di atas seringkali tidak dilakukan satu kali, tapi berkali-kali sehingga membentuk suatu siklus pemrograman (lihat Gambar 3.2).



Gambar 3.2 Siklus Membuat Program

Dalam latihan di buku Informatika Kelas X, kamu mungkin merasakan bahwa program yang kamu buat masih jauh lebih sederhana dibandingkan dengan program yang biasa kamu gunakan dalam kehidupan sehari-hari. Hal tersebut sangatlah wajar, karena program-program yang kompleks dengan tampilan visual dan interaktif yang banyak dipakai saat ini telah melewati tahapan pengembangan yang panjang dan telah melalui beberapa versi. Apabila kamu menggunakan sebuah program yang disediakan secara daring (misalnya untuk berbelanja) dan rutin dalam waktu lama, kamu akan merasakan bahwa program tersebut senantiasa mengalami perubahan dan perbaikan atau penambahan fitur secara terus menerus. Hal ini menjadi bagian dari “siklus kehidupan” suatu program agar terus relevan dengan kebutuhan penggunaannya.

Pendekatan siklus ini juga bermanfaat jika kamu ingin membuat sebuah program yang kompleks. kamu dapat menetapkan suatu bentuk akhir dari program yang akan kamu buat dan mendekomposisinya ke dalam program-program yang lebih sederhana. Misalnya, kamu ingin membuat program yang mampu memprediksi kondisi hutan Indonesia di masa depan. Untuk membuat program yang kompleks tadi, kamu dapat mulai dengan membuat program yang dapat membaca dan mengolah data hutan dalam bentuk tabel terlebih dahulu, kemudian menambahkan analisis data sederhana dalam bentuk grafik batang, kemudian mengubahnya menjadi visualisasi berbentuk peta, baru akhirnya menambah kemampuan prediksi keadaan hutan.

Praktik Baik Pemrograman

Kamu perlu mengumpulkan dengan rapi hasil pekerjaan dari langkah di atas. Misalnya, coret-coretan, algoritma yang dibuat, kode sumber, serta proses pengujian yang dilakukan. Hal ini disebut dokumentasi program. Dokumentasi berguna bagi kamu maupun orang lain untuk memahami kode program dengan cepat. Dokumentasi dapat ditulis baik di dalam kode program maupun di dokumen terpisah (untuk program yang kompleks). Dokumentasi yang ditulis menyatu dengan kode dapat ditulis menggunakan komentar, tetapi komentar tidak perlu menjelaskan setiap baris kode, cukup menjelaskan abstraksi/ide dari kode. Ingatlah bahwa kode program yang ditulis dengan baik sudah pasti akan mudah dipahami.

Contoh Kasus Siklus Pemrograman

Selanjutnya mari kita coba lakukan keempat langkah tersebut untuk membuat sebuah program yang dapat membantu seseorang mengidentifikasi jenis segitiga berdasarkan panjang ketiga sisinya. Seperti kita ketahui, ada beberapa jenis segitiga berdasarkan kondisi panjang masing-masing sisinya, misalnya, segitiga sama sisi, segitiga sama kaki, atau pun segitiga sembarang. Bayangkan sebuah program yang dapat menentukan apakah sebuah segitiga termasuk segitiga sama kaki, sama sisi, atau bukan keduanya ketika diberikan data berupa ketiga panjang sisi-sisi dari segitiga yang dimaksud. Misalnya, jika diberikan masukan berupa panjang sisi-sisi: 2, 2 dan 3, maka program tersebut tentunya harus menghasilkan keluaran “segitiga sama kaki”.

1. Menganalisis Permasalahan

Deskripsi di atas sangat abstrak dan luas. Oleh karena itu, sebagai pemrogram kita perlu menganalisis persoalan dengan mendefinisikan ruang lingkup permasalahan yang diberikan. Definisi ini perlu dibuat dengan baik karena pengecekan kebenaran program akan bergantung pada definisi permasalahan yang telah dibuat. Untuk menggali kebutuhan pembuatan program, kalian dapat bertanya kepada pembuat soal, kepada guru, atau kepada orang-orang yang nantinya akan menggunakan program kalian.

Pada deskripsi mengidentifikasi jenis segitiga di atas, kita perlu mencari jawaban dari beberapa pertanyaan berikut:

- Bagaimana caranya mengidentifikasi data panjang sisi sisi tersebut menunjukkan sisi sebuah segitiga? (karena bisa jadi tidak ada segitiga yang memiliki panjang sisi-sisi sebagaimana data yang diberikan)
- Bagaimana menentukan jenis segitiga dari panjang sisi-sisinya?
- Jenis segitiga apa saja yang harus kita identifikasi (misalnya, apakah kita harus bisa mengidentifikasi segitiga siku-siku?)
- Batasan seperti apa yang harus dipenuhi oleh data masukan?
- Keluaran seperti apa yang harus diberikan oleh program?

Setelah pertanyaan-pertanyaan tersebut terjawab, kamu dapat membuat definisi permasalahan yang lebih formal. Contoh berikut dapat menjadi salah satu cara untuk mendefinisikan permasalahan ini setelah kamu mendapatkan jawaban dari pertanyaan-pertanyaan tersebut.

Deskripsi:

Diberikan tiga buah bilangan bulat yang berada pada rentang $[1, 1000]$ yang merupakan panjang sisi dari sebuah segitiga. Identifikasi apakah ketiga sisi tersebut membentuk segitiga sama sisi, atau segitiga sama kaki, segitiga sembarang, atau tidak bisa membentuk segitiga!

Masukan:

Masukan terdiri atas tiga bilangan bulat a , b , dan c yang merupakan panjang masing-masing sisi segitiga pada rentang $[1, 1000]$

Proses:

Dari masukan yang diberikan, kita harus menentukan terlebih dahulu, apakah ada segitiga dengan panjang sisi-sisi a , b dan c . Jika tidak ada, maka program dapat berhenti dan melaporkan bahwa data masukan tidak menunjukkan sisi-sisi sebuah segitiga. Jika ternyata a , b dan c merupakan sisi-sisi sebuah segitiga, maka program harus menentukan jenis segitiga apakah yang memiliki panjang sisi-sisi a , b dan c tersebut.

Keluaran:

Keluaran berupa teks sebagai berikut ini:

- “*Segitiga Sama Sisi*” jika masukan berupa segitiga sama sisi.
- “*Segitiga Sama Kaki*” jika masukan berupa segitiga sama kaki.
- “*Segitiga Sembarang*” jika masukan berupa segitiga sembarang.
- “*Bukan Segitiga*” jika masukan bukan berupa segitiga.

2. Merancang Solusi

Setelah definisi persoalan (masukan, proses, keluaran) dibuat seperti di atas. Selanjutnya kamu akan mencoba menyusun algoritma untuk menyelesaikan permasalahan tersebut. Pertama-tama, untuk menentukan apakah ada segitiga yang memiliki sisi-sisi dengan panjang sesuai masukan, yaitu a , b dan c , maka nilai-nilai tersebut haruslah memenuhi aturan “Teorema Pertidaksamaan Segitiga” berikut.

“Untuk semua segitiga dengan panjang a , b , dan c , maka haruslah berlaku $a + b > c$ ”

Jika kita terjemahkan teorema di atas untuk sembarang masukan a , b , dan c yang merupakan panjang sisi-sisi segitiga, kita harus memeriksa 3 buah kondisi berikut:

- sisi pertama : $b + c > a$
- sisi kedua : $a + c > b$
- sisi ketiga : $a + b > c$

Ketiga kondisi tersebut harus terpenuhi, agar a , b , dan c dapat membentuk segitiga. Dengan kata lain, jika setidaknya salah satu kondisi tersebut tidak dipenuhi, maka ketiga sisi tersebut tidak bisa membentuk segitiga. Jika hal ini terjadi, program dapat melaporkan hasil ini dan kemudian langsung berhenti.

Jika tidak, berarti a , b , dan c memang benar merupakan sisi-sisi sebuah segitiga, dan program dapat mengidentifikasi jenis segitiga yang sesuai. Selanjutnya, untuk mengidentifikasi jenis segitiga, kita dapat merancang aturan-aturan sebagai berikut:

- Jika tiga panjang sisi sama, maka segitiga tersebut adalah sama sisi;
- Jika hanya dua panjang sisi sama, maka segitiga tersebut sama kaki;
- Jika ketiga panjang sisi berbeda, maka segitiga tersebut adalah segitiga sembarang.

3. Mengimplementasikan Solusi dalam Bentuk Program (Coding)

Selanjutnya, kamu dapat mengimplementasikan algoritma tersebut ke dalam suatu program komputer. Misalnya, kita dapat mencoba implementasikan dalam bahasa pemrograman C. Tentunya, kamu juga dapat menuliskan algoritma di atas dalam bahasa pemrograman lainnya yang telah kamu pelajari seperti Python, atau bahkan menggunakan pemrograman blok.

Catatan: Saat menuliskan kode yang ada pada buku ini, kamu tidak perlu menuliskan nomor baris pada program kamu. Nomor baris pada buku ini ditambahkan untuk mempermudah kamu dalam membaca kode sumber.

```

1  /* program menentukan jenis segitiga */
2  #include <stdio.h>
3  int main()
4  {   int a, b, c;
5      printf("Panjang sisi pertama : "); scanf("%d",&a);
6      printf("Panjang sisi kedua   : "); scanf("%d",&b);
7      printf("Panjang sisi ketiga  : "); scanf("%d",&c);
8
9      /* mengecek teorema pertidaksamaan segitiga */
10     if(((b+c)>a) && ((a+c)>b) && ((a+b)>c))
11     {   if((a==b)&&(b==c))
12         printf("Segitiga Sama Sisi");
13         else if((a==b)||(b==c)||(a==c))
14             printf("Segitiga Sama Kaki");
15         else
16             printf("Segitiga Sembarang");
17     }
18     else
19         printf("Bukan segitiga");
20
21     return 0;
22 }

```

Gambar 3.3 Gambar Kode Program Identifikasi Jenis Segitiga

Sumber: Dela Chaerani/Kemendikbudristek (2024)

Penjelasan Program:

Jika kamu perhatikan program di atas, ekspresi pada baris 9 digunakan untuk mulai mengecek “**Teorema Pertidaksamaan Segitiga**” sesuai dengan ekspresi pada tahap merancang solusi. Perhatikan baris 10 pada program di atas, pengecekan dilakukan dengan memanfaatkan operator **AND** (&&). Untuk mengetahui mengenai penggunaan operator pada pemrograman C, kamu dapat membaca kembali pelajaran kelas X.

4. Menguji Program

Setelah program selesai, kita perlu menguji kebenaran program tersebut. Ada banyak metode formal untuk menguji suatu program, tetapi yang akan dicontohkan adalah menguji program menggunakan kasus uji seperti yang telah diberikan di kelas X. Misalnya, kita akan menguji kode program di atas dengan menggunakan pasangan kasus uji pada tabel berikut:

Tabel 3.1 Lembar Kerja untuk Menguji Program Jenis Segitiga

No	Masukan			Keluaran Seharusnya	Keluaran Program	Hasil Benar?
	a	b	c			
1	100	100	100	segitiga sama sisi		
2	1000	1000	500	Segitiga sama kaki		
3	100	200	500	Bukan segitiga		
4	200	300	400	Segitiga sembarang		

Apabila terdapat kasus uji yang tidak dapat menghasilkan keluaran yang benar oleh program, lakukan analisis kembali untuk mengidentifikasi kesalahan tersebut dan memperbaiki program kamu hingga benar.

Ini Informatika

Dari ilustrasi di atas, kamu telah merasakan bahwa dalam membuat program, kamu perlu memahami permasalahan yang kamu kerjakan. Semakin kompleks suatu permasalahan yang harus diselesaikan, atau semakin banyak fitur yang harus diimplementasikan, program juga akan menjadi semakin kompleks dan mungkin menjadi panjang (lebih banyak instruksinya). Oleh karena itu, tim yang membuat program akan terdiri atas banyak orang dengan keahlian dan spesialisasi yang berbeda. Di antara mereka juga ada orang-orang yang ahli dan paham dengan permasalahan yang diselesaikan.



Mengamati Evolusi Program

Jenis Aktivitas: Individu

No Aktivitas: SAP-K11-01

Deskripsi Tugas:

Pada bagian ini telah dijelaskan bahwa suatu program dikembangkan bertahap dalam suatu siklus. Hal ini juga berlaku pada program, baik berbentuk aplikasi maupun web yang kamu gunakan. Sekarang pilihlah satu aplikasi atau web yang sering kamu gunakan, kemudian buatlah infografik linimasa (*timeline*) sederhana yang menunjukkan proses perubahan program seiring dengan berjalannya waktu. Kamu dapat melakukan pencarian untuk menemukan versi lama dari program. Untuk web, kamu dapat menggunakan aplikasi web seperti web.archive.org yang dapat menyimpan versi lama dari web secara periodik.

B. Strategi Algoritmik dan Pemrograman

Membuat Algoritma tidak terlepas dari berpikir secara komputasional (*computational thinking*). Berpikir komputasional ini merupakan suatu kerangka dan proses berpikir yang mencakup perangkat keras, perangkat lunak, dan menalar (*reasoning*) mengenai sistem dan persoalan. Moda berpikir (*thinking mode*) ini didukung dan dilengkapi dengan pengetahuan teoritis dan praktis, serta teknik untuk menganalisis, memodelkan dan menyelesaikan persoalan. Berikut ini pembahasan beberapa konsep dan strategi berpikir komputasional yang biasa digunakan dalam menyelesaikan persoalan komputasi.

1. Rekursi

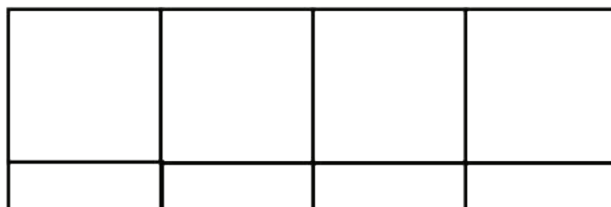
Pada bagian ini kamu akan mempelajari tentang konsep rekursi dan beberapa contoh permasalahan yang dapat diselesaikan dengan menggunakan konsep tersebut, salah satunya adalah **barisan Fibonacci**.



Gambar 3.4 Ilustrasi Rekursi

Suatu masalah dapat didekomposisi menjadi permasalahan yang serupa, tetapi ukurannya lebih kecil. Saat kita diminta untuk memindahkan satu kardus buku yang sangat berat dan tidak dapat kita angkat, kita akan membagi kardus tersebut ke dalam beberapa kardus yang lebih ringan sehingga pekerjaan tersebut menjadi lebih mudah untuk dikerjakan. Ketika menghitung suatu nilai faktorial, kita pun harus menghitung nilai faktorial yang lebih kecil. Misalnya, ketika menghitung 10 faktorial, kita juga harus menyelesaikan 1 faktorial, 2 faktorial, hingga 9 faktorial terlebih dahulu.

Secara alami, terdapat banyak permasalahan yang dapat dimodelkan dengan lebih mudah menggunakan konsep rekursif ini. Pada bagian ini, kamu akan mempelajari konsep dasar rekursi yang akan sangat berguna untuk melakukan dekomposisi pada suatu permasalahan besar dalam bentuk permasalahan yang lebih kecil dan lebih mudah untuk diselesaikan. Rekursi didefinisikan sebagai “sesuatu” yang mengandung “sesuatu” itu sendiri. Dapatkah kamu melihat rekursi dari gambar-gambar berikut ini?



Gambar 3.5 Beberapa Contoh Rekursi: Segitiga Sierpinski (Kiri), Boneka Bersarang dari Rusia (Kanan)

Sumber: Siti Maesaroh/Kemendikbudristek (2024); Fanghong/Wikimedia Commons (2005)

Dalam pembahasan kali ini, kita akan membahas fungsi/barisan rekurensi (*recurrence*) yaitu fungsi/barisan yang nilai dari fungsi atau barisannya ditentukan/tergantung dari nilai fungsi/barisan itu sendiri secara rekursif pada urutan nilai-nilai sebelumnya. Misalnya, kita memiliki sebuah barisan $a_i, i=1,2,\dots,n$ sebagai berikut:

$$\{a_i\} = 1, 3, 5, 7, \dots$$

Nilai pertama dari barisan (a_1) adalah 1. nilai-nilai berikutnya dalam barisan tersebut dihitung dengan cara menambahkan nilai 2 kepada nilai barisan sebelumnya. Kita dapat menuliskan dalam notasi rekursif sebagai berikut:

$$a_i = \begin{cases} 1, & \text{jika } i = 1 \\ a_{i-1} + 2, & \text{jika } i > 1 \end{cases}$$

Pada definisi sebuah barisan/fungsi rekursif, selalu ada minimal dua hal yang harus ditentukan:

- **Basis:** menunjukkan dasar/nilai awal dari fungsi/barisan tersebut. Misalnya, pada contoh di atas, $a_1=1$
- **Rekursi:** menunjukkan hubungan antara nilai dari fungsi/barisan tersebut dengan nilai-nilai sebelumnya yang telah diketahui. Misalnya, pada contoh di atas: $a_i=a_{i-1}+2$, jika $i > 1$.

Sebuah fungsi/barisan rekursif bisa jadi ditentukan dari tidak hanya satu buah nilai sebelumnya saja, tetapi dapat juga dari 2, 3, ... dan seterusnya, nilai sebelumnya. Sebagai contoh sebuah barisan dapat didefinisikan sebagai berikut:

$$a_i = \begin{cases} 1, & \text{jika } i = 1 \text{ atau } i = 2 \\ a_{i-1} + a_{i-2} & \text{jika } i > 2 \end{cases}$$

Barisan ini dimulai dengan nilai 1, kemudian untuk menentukan nilai berikutnya, kita hitung dengan cara menjumlahkan dua nilai sebelumnya pada barisan tersebut, sehingga didapatkan barisan sebagai berikut:

$$\{a_i\} = 1, 1, 2, 3, 5, 8, 13, 21, \dots$$

Barisan di atas biasa disebut sebagai barisan Fibonacci yang dipopulerkan oleh seorang matematikawan yang berasal dari Italia bernama *Fibonacci* (nama lengkap Leonardo Bonacci, 1170 – 1250 M). Perlu diperhatikan bahwa karena pada bagian rekursi, kita memerlukan dua nilai terakhir, pada bagian

basis, kita perlu mendefinisikan dua nilai pertama dari barisan tersebut. Secara umum, banyaknya nilai yang harus didefinisikan pada bagian basis ditentukan oleh banyaknya suku barisan yang diperlukan pada bagian definisi rekursi.



Gambar 3.6 Kode Program *Fibonacci*

Sumber: Dela Chaerani/Kemendikbudristek (2024)



Ayo Berlatih

Memahami Relasi Rekurensi

Jenis Aktivitas: Individu

No Aktivitas: SAP-K11-02

Deskripsi Tugas:

Relasi rekurensi (*recurrence relation*) adalah sebuah tipe relasi matematis yang definisi sebuah fungsi atau barisannya dinyatakan secara rekursif, artinya merujuk pada fungsi atau barisan itu sendiri. Pada bagian ini, kamu akan berlatih untuk memahami definisi relasi rekurensi dan bagaimana menerapkannya, serta membuat definisi rekursif dari sebuah permasalahan.

1. Tentukan suku ke-10 dari barisan yang didefinisikan sebagai berikut:

$$a_i = \begin{cases} 1 & \text{jika } i = 1 \text{ atau } i = 2 \\ 2a_{i-1} + 2a_{i-2} & \text{jika } i > 2 \end{cases}$$

atau dengan kata lain, barisan tersebut dimulai dengan nilai 1, 1, kemudian untuk menghitung suku berikutnya, kita jumlahkan antara suku sebelumnya dengan dua kali dari suku sebelum suku sebelumnya.

2. Faktorial dari sebuah bilangan bulat $n \geq 1$, ditulis sebagai $n!$, didefinisikan sebagai sebuah nilai yang dihitung dengan mengalikan semua bulat dari 1 sampai dengan n .

Contohnya, faktorial dari 5 adalah $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Buatlah sebuah rekursif untuk menghitung nilai $n!$



Ayo Berlatih



Menerapkan Konsep Rekursi

Jenis Aktivitas: Individu

No Aktivitas: SAP-K11-03

Deskripsi Tugas:

Selesaikanlah dua *problem* berikut dengan menerapkan konsep rekursi yang telah kamu pelajari. Setelah mengerjakan *problem* tersebut, diskusikanlah solusinya dengan temanmu.

Permasalahan 1: Memasang Keramik



Lantai



Keramik

Terdapat sebuah lantai yang berukuran $2 \times N$. Pada lantai tersebut, ingin dipasang N buah keramik, yang masing-masing berukuran 1×2 (perhatikan Gambar 3.7).

Gambar 3.7 Ilustrasi Memasang Keramik

Setiap keramik dapat dipasang secara mendatar (horizontal) maupun secara tegak (vertikal). Sebagai contoh, jika $N=4$, maka akan ada 5 buah cara berbeda memasang 4 keramik, yaitu sebagai berikut.

- Semua keramik dipasang secara horizontal.
- Semua keramik dipasang secara vertikal.
- Paling kiri ada 2 keramik vertikal, sisanya horizontal.
- Paling kiri ada 2 keramik horizontal, sisanya vertikal.
- Paling kiri dan paling kanan vertikal, sisanya horizontal.

Tentukan ada berapa cara memasang keramik untuk $N=8$?

Permasalahan 2: Menumpuk Panekuk

Budi memiliki setumpuk panekuk (*pancake*) yang memiliki ukuran dari besar sampai kecil. Kue panekuk tersebut ditumpuk di atas sebuah piring dengan aturan bahwa panekuk yang besar harus berada di bawah panekuk yang lebih kecil.



Gambar 3.8 Ilustrasi Kue Panekuk

Budi ingin memindahkan panekuk ini dari satu piring ke piring lainnya, namun dalam prosesnya ia tetap ingin mengikuti aturan bahwa panekuk yang besar harus selalu berada di bawah panekuk yang lebih kecil. Selain itu, Budi juga hanya boleh memindahkan satu buah panekuk saja, pada satu waktu tertentu, dari satu piring ke piring lainnya. Budi menyadari bahwa ia memerlukan sebuah piring tambahan untuk dapat melakukan perpindahan ini. Jika piring asal panekuk diberi nama A, piring tujuan diberi nama C, maka Budi akan menyiapkan sebuah piring bantuan sebagai tempat sementara yang diberi nama piring B.

Budi ingin mengetahui berapa banyak langkah pemindahan panekuk yang harus ia lakukan untuk dapat memindahkan semua panekuk yang dimilikinya, dari piring A ke piring C (dengan menggunakan piring B sebagai tempat sementara). Misalnya, jika Budi memiliki 2 buah panekuk saja (diberi nama panekuk 1 yang berukuran lebih kecil dari panekuk 2), maka ia akan membutuhkan minimal 3 langkah pemindahan:

1. Pindahkan panekuk 1 dari piring A ke piring B
2. Pindahkan panekuk 2 dari piring A ke piring C
3. Pindahkan panekuk 1 dari piring B ke piring C

Berapakah jumlah langkah minimal yang diperlukan apabila Budi memiliki 6 buah panekuk?

2. Algoritma *Greedy*

Greedy secara harfiah berarti rakus atau tamak. Meskipun dalam pengertian sehari-hari, kata *rakus* dan *tamak* memiliki konotasi negatif, tetapi dalam konteks informatika, kita mengartikan *greedy* dalam konteks sebagai sebuah strategi penyelesaian masalah yang dapat berguna dalam merancang sebuah algoritma atau solusi bagi sebuah permasalahan komputasional. Oleh karena itu, diharapkan tidak ada konotasi negatif pada kata *greedy* dalam konteks ini.

Teknik *greedy* adalah salah satu teknik penyelesaian masalah yang biasa digunakan untuk menyelesaikan permasalahan optimasi. Permasalahan optimasi berarti kita ingin menghitung sebuah hasil yang terbaik dari sebuah proses tertentu. Terbaik yang dimaksud dapat berarti nilai yang paling kecil ataupun paling besar, tergantung dari jenis permasalahannya. Dalam menyelesaikan permasalahan optimasi seperti ini, algoritma *greedy* akan menerapkan prinsip “mengambil serangkaian langkah terbaik pada setiap saat”.

Contoh Kasus 1: Membawa Ikan 1

Budi ingin membawa beberapa ekor ikan yang sudah tersimpan dalam kantong-kantong plastik untuk diangkut ke dalam mobilnya. Terdapat 8 buah kantong dengan yang berisi masing-masing 6, 6, 3, 5, 2, 8, 4, dan 3

ekor ikan. Namun sayangnya, mobilnya hanya mampu membawah 4 buah kantong. Kantong-kantong manakah yang harus dibawa oleh Budi agar dapat membawa ikan sebanyak mungkin?

Jawab:

Untuk dapat membawa sebanyak mungkin ikan, Budi harus memilih kantong-kantong dengan sebanyak mungkin ikan. Oleh karena itu, algoritma *greedy* dapat diterapkan pada kasus ini, dengan cara kita mengambil kantong mulai dari yang berisi ikan paling banyak terlebih dahulu sampai didapatkan 4 buah kantong. Dengan demikian, kita harus mengurutkan kantong-kantong terlebih dahulu mulai dari yang paling banyak ikannya, sampai dengan yang paling sedikit, sehingga urutannya menjadi 8, 6, 6, 5, 4, 3, 3, 2. Jika kita ambil 4 buah kantong pertama, maka total banyaknya ikan yang dapat dibawa adalah $8 + 6 + 6 + 5 = 25$ ekor ikan. Tentunya tidak ada pilihan 4 kantong yang akan menghasilkan total banyaknya ikan lebih dari 25 ekor.

```
1  /* Program Mengambil 4 data terbesar */
2  #include <stdio.h>
3  main()
4  {   int sorting;
5      int data=8;
6      int ikan[data]={6,6,3,5,2,8,4,3};
7      for(int c=1;c<data;c++)
8      {
9          for(int d=0;d<(data-c);d++)
10         {
11             if(ikan[d]<ikan[d+1])
12             {
13                 sorting=ikan[d];
14                 ikan[d]=ikan[d+1];
15                 ikan[d+1]=sorting;
16             }
17         }
18     }
19     int jumlah=0;
20     for(int i=0;i<(data-4);i++)
21     {
22         jumlah=jumlah+ikan[i];
23     }
24     printf("Jumlah ikan yang dibawa : %d",jumlah);
```

Gambar 3.9 Program Contoh Kasus 1 - Algoritma *Greedy*

Sumber: Dela Chaerani/Kemendikbudristek (2024)

Contoh Kasus 2: Membawa Ikan 2

Kali ini, Budi harus membawa sedikitnya 15 ekor ikan. Tentukan jumlah kantong terkecil yang harus dibawa oleh Budi agar minimal terdapat 15 ekor ikan yang terbawa.

Jawab:

Sama seperti pada permasalahan sebelumnya, kita dapat menerapkan algoritma *greedy* untuk menyelesaikan permasalahan ini. Dalam hal ini, untuk memperkecil banyaknya kantong yang harus dibawa, kita juga selalu memilih kantong dengan jumlah ikan terbanyak terlebih dahulu. Jika kita memilih kantong dengan jumlah ikan = 8 dan 6, maka kita sudah memiliki 14 ekor ikan. Selanjutnya, kita hanya perlu mengambil 1 kantong lagi (yang mana saja) agar total jumlah ikan menjadi lebih dari 15. Oleh karena itu, jawaban yang diinginkan adalah 3 buah kantong. Jelas bahwa tidak ada pilihan yang memungkinkan kita mendapatkan 15 ekor ikan dengan 2 kantong atau kurang.

Pada kedua contoh di atas, terdapat satu langkah yang penting yang biasa diterapkan pada penyelesaian masalah secara *greedy*, yaitu proses mengurutkan sebuah data agar menjadi terurut (mungkin dari kecil ke besar atau sebaliknya), agar kemudian kita dapat melakukan serangkaian pengambilan langkah secara *greedy* pada data yang sudah terurut tersebut. Pola seperti ini umum digunakan pada penyelesaian permasalahan secara *greedy*.



Ayo Berlatih



Mengerjakan Pekerjaan Rumah (PR)

Jenis Aktivitas: Kelompok

No Aktivitas: SAP-K11-04

Deskripsi Tugas

Cici menerima 10 buah pekerjaan rumah (PR) yang harus ia kerjakan. Setelah melihat isi dari masing-masing PR, Cici memiliki perkiraan, berapa lama waktu yang diperlukan untuk mengerjakan masing-masing PR tersebut, seperti terlihat pada tabel di bawah ini.

No.	PR	Waktu pengerjaan (jam)
1.	A	1.5
2.	B	3
3.	C	1
4.	D	0,5
5.	E	4

No.	PR	Waktu pengerjaan (jam)
6.	F	1
7.	G	2,5
8.	H	1
9.	I	0,5
10.	J	2

Sayangnya, ia tidak punya banyak waktu untuk mengerjakan semua PR. Cici menghitung bahwa ia hanya punya waktu total = 8 jam sebelum semua PR tersebut harus dikumpulkan. Cici ingin menentukan, PR mana yang harus ia kerjakan terlebih dahulu, dengan pertimbangan bahwa setiap PR memiliki nilai yang sama besarnya (terhadap nilai akhir Cici). Bantulah Cici menentukan PR yang mana saja yang harus ia kerjakan dalam waktu maksimal 8 jam untuk mendapatkan total nilai akhir yang sebesar-besarnya.



Ayo Berlatih



Mengunjungi Kebun Binatang

Jenis Aktivitas: Individu

No Aktivitas: SAP-K11-05

Deskripsi Tugas

Dina sedang bertamasya mengunjungi kebun binatang. Setiap hari, kebun binatang mengadakan beberapa pertunjukan atraksi hewan yang dapat ditonton oleh para pengunjung. Berikut adalah jadwal yang telah ditetapkan oleh pengelola kebun binatang.

No.	Waktu Mulai	Waktu Selesai	Pertunjukan/Atraksi Hewan
1.	09:15	10:30	Orang Utan
2.	08:00	09:30	Penguin
3.	10:00	12:00	Harimau

No.	Waktu Mulai	Waktu Selesai	Pertunjukan/Atraksi Hewan
4.	13:00	14:30	Beruang Madu
5.	11:00	12:30	Burung Pemangsa
6.	14:00	15:00	Buaya
7.	15:30	16:30	Panda
8.	16:00	17:00	Ular Piton
9.	15:00	15:30	Singa
10.	15:30	16:00	Anjing Laut

Tentunya dalam satu waktu tertentu, Dina hanya dapat menonton satu pertunjukan atraksi hewan. Dina ingin dapat melihat spertunjukan dalam satu hari tersebut sebanyak-banyaknya. Ia tidak memiliki preferensi dalam melihat pertunjukan hewan (semuanya ia anggap sama menariknya). Tentukan ada berapa banyak maksimal pertunjukan yang dapat ditonton oleh Dina?



Ayo Berlatih



Menukarkan Uang

Jenis Aktivitas: Individu

No Aktivitas: AD-K11-06

Deskripsi Tugas

Dalam kehidupan sehari-hari, kita pasti sudah banyak terbiasa dengan perhitungan yang melibatkan uang. Misalnya, ketika kamu membeli sebuah barang/makanan, ataupun ingin membayar untuk sebuah jasa tertentu, kita seringkali menyiapkan sejumlah uang tertentu sesuai dengan harga barang atau jasa tersebut. Selanjutnya, bagi penjual atau penyedia jasa, apabila mereka menerima uang pembayaran dengan jumlah total yang lebih besar dari harga yang ditetapkan, mereka pun harus menyiapkan uang kembalian sesuai dengan jumlah kelebihan pembayaran.

Di Indonesia, mata uang rupiah memiliki beberapa pecahan uang, mulai dari yang terkecil Rp100, Rp200, Rp500, sampai dengan Rp100.000. Seandainya kita memiliki sejumlah pecahan uang, misalnya beberapa uang seribuan, dua ribuan, lima ribuan, sepuluh ribuan dan dua puluh ribuan. Jika kita ingin mendapatkan uang tepat sejumlah Rp38.000, maka kita dapat memilih beberapa cara berikut.

- 3 lembar sepuluh ribuan, ditambah 1 lembar lima ribuan, ditambah 2 lembar seribuan, ditambah 2 koin lima ratus, dengan total ada 8 buah lembaran uang/koin
- 1 lembar dua puluh ribuan, ditambah 1 lembar sepuluh ribuan ditambah 4 lembar dua ribuan, totalnya menjadi 6 lembaran uang
- 1 lembar dua puluh ribuan, ditambah 1 lembar sepuluh ribuan, ditambah 1 lembar lima ribuan ditambah 1 lembar dua ribuan, ditambah 1 lembar seribuan, dengan total ada 5 lembaran uang.

Jelas bahwa jumlah total lembaran yang dibutuhkan tergantung dari pemilihan pecahan uang yang kita gunakan. Nah, permasalahan yang mungkin kita tanyakan adalah bagaimana caranya memilih pecahan-pecahan uang yang akan digunakan sedemikian rupa sehingga total lembaran yang diperlukan untuk menghasilkan suatu nilai uang tertentu menjadi sekecil mungkin?

Pada contoh di atas, dapat diperiksa bahwa untuk menghasilkan nilai uang sebesar tiga puluh delapan ribu rupiah dari pecahan-pecahan seribuan, dua ribuan, lima ribuan, sepuluh ribuan, dan dua puluh ribuan diperlukan minimal 5 buah lembar, yaitu sesuai dengan cara terakhir di atas.

Dapatkah kamu mencari strategi yang umum untuk menyelesaikan permasalahan serupa jika jumlah nilai uang yang dihasilkan berbeda (tetapi dengan pecahan-pecahan uang yang sama)? Kita bisa menganggap bahwa jumlah nilai yang diinginkan selalu merupakan kelipatan ribuan rupiah (sehingga selalu bisa didapatkan dengan menggabungkan pecahan-pecahan di atas).

3. Pemrograman Dinamis

Saat menyelesaikan sebuah permasalahan optimasi (mencari nilai terbesar/terkecil), terkadang kita harus memperhitungkan beberapa kemungkinan pengambilan langkah untuk menyelesaikan permasalahan tersebut. Kemungkinan-kemungkinan tersebut mungkin memiliki akibat/konsekuensi terhadap langkah-langkah selanjutnya sehingga pendekatan seperti teknik *greedy* mungkin tidak akan menghasilkan jawaban yang optimal. Dalam hal ini, teknik pemrograman dinamis atau *dynamic programming* (DP) mungkin akan lebih sesuai diterapkan. Teknik DP mengandung dua unsur utama, yaitu:

- a. Optimasi (mencari nilai terkecil/terbesar) melalui serangkaian pilihan. Serupa dengan teknik *greedy*, kita harus menentukan rangkaian langkah apa yang akan menghasilkan nilai optimal di akhir. Namun, berbeda dengan permasalahan yang dapat diselesaikan dengan teknik *greedy*, permasalahan yang sesuai untuk teknik DP memiliki struktur sedemikian rupa sehingga pilihan langkah terbaik saat ini belum tentu merupakan pilihan terbaik secara keseluruhan, sehingga prinsip *greedy* belum tentu dapat diterapkan, dan semua kemungkinan kombinasi pilihan langkah harus diperhitungkan.
- b. Nilai optimal yang diinginkan untuk permasalahan tersebut biasanya dapat dinyatakan sebagai kombinasi optimal dari sub-sub permasalahan yang sama, tetapi dengan ukuran yang lebih kecil (atau dengan kata lain, dapat dinyatakan secara rekursif). Namun, sub-sub permasalahan yang harus dipertimbangkan, biasanya memiliki *overlap* (persinggungan) sehingga dalam proses perhitungannya, diperlukan cara yang efisien untuk menghitung solusi untuk sub-sub permasalahan yang diperlukan, agar tidak terjadi perulangan/duplikasi dalam proses perhitungan. Cara yang umum digunakan adalah dengan menyimpan semua solusi dari *subproblem* yang sudah diketahui dalam sebuah tempat penyimpanan/tabel. Teknik ini biasa disebut sebagai teknik memoisasi.

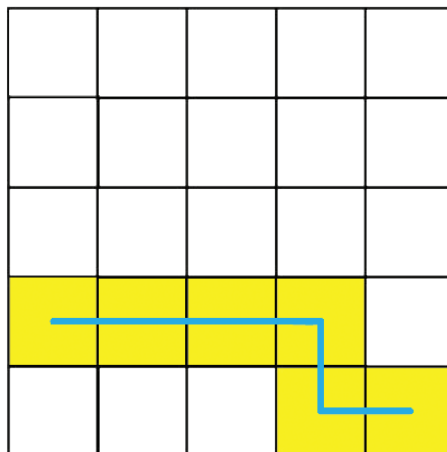
Contoh Kasus: Memanen Tanaman Cabai

Agria ingin memanen tanaman cabai di halaman rumahnya. Tanaman tersebut ditata dalam bentuk kotak-kotak persegi seperti ilustrasi di bawah ini. Angka pada setiap kotak mewakili jumlah cabai yang ada di masing-masing tanaman.

0	1	2	3	10
10	2	10	10	5
5	3	6	5	0
0	4	0	2	3
20	5	1	0	0

Gambar 3.10 Ilustrasi Jumlah Cabai yang Dipanen Agria

Agria tidak punya waktu banyak karena ia harus segera pergi ke kampus. Oleh karena itu, ia tidak bisa memetik seluruh cabai tersebut. Ia hanya bisa mulai dari kotak manapun di kolom paling kiri, dan berhenti di kotak manapun di kolom paling kanan. Agria hanya bisa bergerak ke kotak di tepat setelah kanannya atau bawahnya. Berikut adalah salah satu dari sekian banyak kemungkinan jalur yang dapat dilalui oleh Agria untuk memetik cabai.

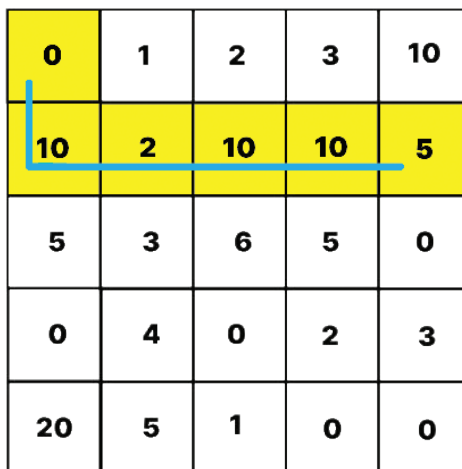


Gambar 3.11 Kemungkinan Jalur Memetik Cabai Agria

Berapakah jumlah cabai terbanyak yang bisa dikumpulkan oleh Agria?

Jawab:

Pertama, perlu dipahami bahwa penggunaan teknik *greedy* pada permasalahan ini tidak akan menghasilkan jawaban yang benar/optimal. Dapat dilihat bahwa jika kita menggunakan prinsip *greedy*, maka kita akan memilih untuk memulai dari kolom pertama baris terakhir, dengan nilai jumlah cabai terbesar, yaitu 20. Namun, jika kita memulai dari sini, maka tidak ada pilihan lain untuk langkah-langkah selanjutnya, selain bergerak terus ke kanan. Maka nilai total cabai yang akan didapatkan adalah $20 + 5 + 1 + 0 + 0 = 26$. Jelas bahwa ada pilihan-pilihan jalur lain yang akan menghasilkan total nilai cabai > 26 , misalnya langkah sebagai berikut akan menghasilkan jumlah total cabai $= 0 + 10 + 2 + 10 + 10 + 5 = 37$.

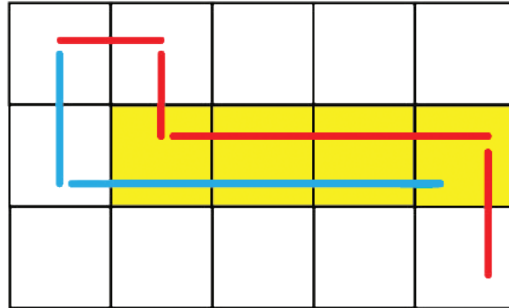


0	1	2	3	10
10	2	10	10	5
5	3	6	5	0
0	4	0	2	3
20	5	1	0	0

Gambar 3.12 Kemungkinan Jalur Lain Memetik Cabai Agria

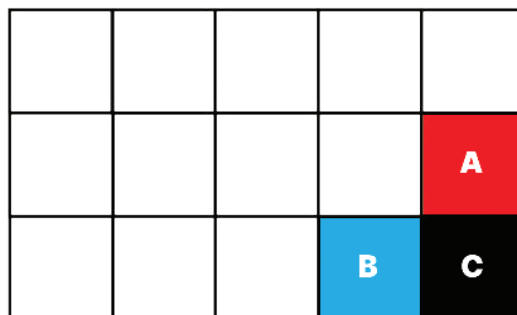
Metode berikutnya yang bisa kita pikirkan solusinya adalah mencoba semua kemungkinan jalur lalu menghitung berapa nilai total cabai yang bisa didapatkan, dan mencari nilai terbesarnya. Namun dengan metode ini kita menemui kendala lainnya, yaitu akan ada terlalu banyak kemungkinan yang harus kita perhitungkan. Satu hal yang dapat kita segera pahami adalah bahwa ada banyak sekali persinggungan di antara jalur-jalur yang berbeda sehingga akan ada banyak sekali perulangan yang tidak perlu ketika kita menghitung nilai total 0 cabai dari semua

kemungkinan jalur yang ada. Sebagai contoh, kedua jalur di bawah ini (jalur biru dan jalur merah) akan melalui empat kotak yang sama dan menghitung penjumlahan dari nilai di keempat kotak yang sama tersebut (warna kuning).



Gambar 3.13 Contoh Ilustrasi Persinggungan antara Dua Kemungkinan Jalur (*Overlap*)

Dengan menggunakan prinsip pemrograman dinamis, yang perlu kita lakukan adalah pertama-tama menyatakan solusi/penyelesaian dari permasalahan awal sebagai kombinasi dari sub-permasalahan yang lebih kecil. Dalam hal ini, kita dapat membuat argumentasi bahwa nilai jumlah cabai terbanyak yang bisa kita kumpulkan sampai dengan suatu kotak tertentu (di mana pun kolomnya) tergantung dari nilai terbaik jumlah cabai sampai dengan kotak di atasnya, atau kotak di sebelah kirinya (jika ada), dan tinggal kita jumlahkan saja dengan nilai banyaknya cabai di kotak akhir tersebut. Ini disebabkan kita hanya bisa bergerak ke kanan atau ke bawah saja. Misalnya, pada Gambar 3.14 ini.



Gambar 3.14 Ilustrasi Prinsip Pemrograman Dinamis pada Persoalan Panen Cabai Agria

Nilai terbaik yang bisa kita dapatkan akan berakhir pada kotak berwarna hitam yang dapat dihitung dengan cara menghitung nilai terbaik yang didapatkan sampai dengan kotak merah (misalkan nilainya = A), dan sampai dengan kotak warna biru (misalkan nilainya = B). Maka, untuk mendapatkan nilai terbaik sampai dengan kotak warna hitam, kita hanya mencari, manakah jumlah yang tertinggi antara nilai A dan B, kemudian nilai tersebut dijumlahkan dengan nilai C.

Proses di atas mengubah permasalahan ini menjadi bersifat rekursif. Kita bisa menggunakan hasil perhitungan pada kotak-kotak sebelumnya untuk menghitung nilai terbaik pada kotak-kotak selanjutnya (yang berada di posisi lebih ke kanan atau ke bawah), sehingga dengan cara ini, kita bisa menghindari perulangan (duplikasi) proses perhitungan. Proses ini biasanya menggunakan sebuah tabel perhitungan yang biasa disebut sebagai tabel memoisasi (atau tabel *Dynamic Programming*). Istilah *memoisasi* berasal dari bahasa latin memorandum yang berarti 'mengingat', yang kemudian biasa disingkat sebagai memo dalam bahasa Inggris. Harap bedakan istilah memoisasi ini dengan memorisasi (*memorization*) yang juga memiliki arti yang serupa (proses mengingat). Memoisasi memiliki arti yang khusus dalam dunia komputasi, yaitu menyimpan/mengingat hasil perhitungan yang telah dilakukan sebelumnya, sehingga tidak perlu mengulang perhitungan yang sama dua kali.

Untuk soal ini, kita buat tabel memoisasi tersebut sebagai berikut.

1. Kotak paling kiri atas kita berikan nilai = nilai isi kotak tersebut (0)
2. Untuk setiap kotak lainnya, misalkan A = nilai yang sudah dihitung pada tabel memoisasi untuk kotak yang ada di atasnya (atau 0 jika kotak saat ini ada di baris teratas), dan B = nilai yang sudah dihitung pada tabel memoisasi untuk kotak yang ada di sebelah kirinya (atau 0 jika kotak saat ini ada di kolom paling kiri), serta misalkan C = nilai cabai yang ada pada kotak saat ini. Maka kita isi kotak saat ini pada tabel memoisasi dengan nilai $\max(A, B) + C$.
3. Kita lakukan proses di atas sampai tabel memoisasi terisi penuh (sesuai ukuran tabel nilai cabai di awal). Nilai paling besar pada tabel memoisasi menunjukkan nilai total jumlah cabai terbesar yang bisa dikumpulkan.

Hasil tabel memoisasi yang sudah terisi penuh untuk soal di atas adalah sebagai berikut.

Tabel 3.2 Memoisasi Persoalan Panen Cabai Agria

0	1	3	6	16
10	12	22	32	37
15	18	28	37	37
15	22	28	39	42
35	40	41	41	42



Ayo Berlatih



Bermain Angka

Jenis Aktivitas: Berpasangan

No Aktivitas: SAP-K11-07

Deskripsi Tugas

Ani dan Budi sedang bermain dengan sebuah permainan angka. Pertama Ani akan memilih sebuah angka bilangan bulat positif n . Selanjutnya, Budi harus mengubah bilangan n ini menjadi angka 1 dengan menerapkan serangkaian langkah sebagai berikut.

1. Budi boleh mengganti bilangan n dengan $n - 1$.
2. Jika bilangan saat ini adalah genap (habis dibagi 2), maka Budi boleh menggantinya dengan $n/2$.
3. Jika bilangan saat ini habis dibagi 3, maka Budi boleh menggantinya dengan $n/3$.

Proses ini harus dilakukan oleh Budi secara terus menerus sampai bilangan yang dimilikinya menjadi 1. Misalnya, jika Ani memilih $n = 5$, maka Budi dapat melakukan proses mengubah 5 menjadi 1 sebagai berikut: $5 \rightarrow 4 \rightarrow 2 \rightarrow 1$ (dalam tiga langkah). Buatlah program untuk menentukan, berapakah jumlah langkah minimum yang diperlukan, jika Ani memilih $n = 25$?

C. Desain Struktur Data

Pada jenjang sebelumnya yaitu di kelas X, kamu telah mempelajari mengenai beberapa bahasa pemrograman, salah satunya adalah bahasa C yang telah dipelajari pada elemen Algoritma dan Pemrograman. Kemampuan bahasa pemrograman terutama bahasa C akan kamu gunakan dalam topik ini. Selanjutnya, untuk lebih meningkatkan pemahaman kamu dalam elemen Algoritma dan Pemrograman, kamu akan mempelajari konsep larik serta karakter dan *string*. Kedua konsep ini sangat penting dalam membuat program dan akan kamu gunakan untuk menyelesaikan berbagai permasalahan yang diberikan pada bagian D dalam bab ini.

1. Larik (Array)

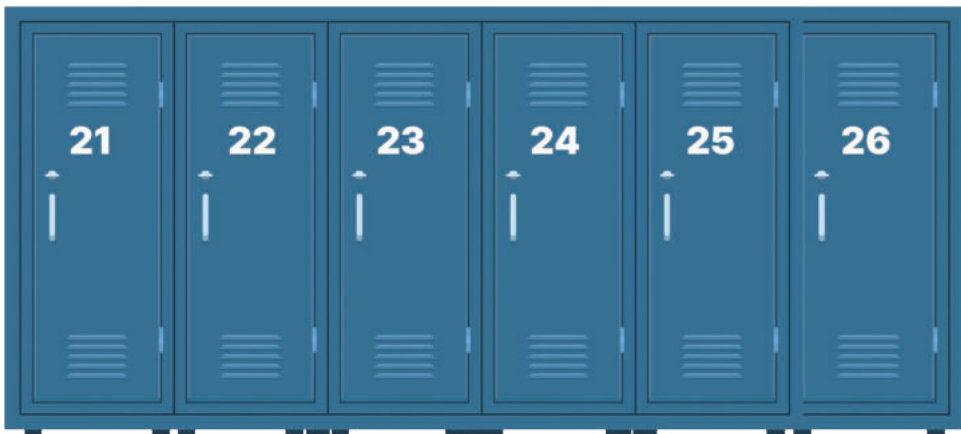
Saat ini mungkin kamu memiliki pertanyaan seperti “mengapa contoh dan permasalahan yang diberikan selama ini adalah hal yang dapat diselesaikan oleh manusia secara manual atau dengan kalkulator?” Jawabannya adalah karena permasalahan/problem tersebut diberikan untuk mendukung proses kamu menguasai kompetensi dalam algoritma dan pemrograman.

Pada praktiknya, program digunakan untuk mengolah data yang berukuran besar dan membutuhkan waktu yang sangat lama jika dikerjakan manual oleh manusia. Misalnya, kamu perlu menghitung statistika deskriptif (seperti rata-rata, nilai minimal, nilai maksimal, standar deviasi, dan sebagainya) dari data seluruh penduduk Indonesia. Walaupun masalahnya sederhana, tetapi karena jumlah data yang diolah sangat banyak dan berukuran besar, waktu pengerjaan pun menjadi sangat lama bagi manusia. Bahkan ada kemungkinan data berukuran besar tersebut tidak dapat diolah menggunakan aplikasi pengolah data (*spreadsheet*) yang tidak dirancang untuk mengolah data sebesar itu. Untuk solusinya, kamu dapat menggunakan program khusus untuk mengolah data berukuran besar atau membuat program sendiri yang mampu menyimpan dan mengolah data berukuran besar.

Kita pun sampai pada pertanyaan besar: bagaimana caranya membuat program yang mampu menyimpan dan mengolah data berukuran besar? Sebelumnya kamu telah mengenal konsep variabel yang mampu menyimpan satu buah nilai dengan tipe data tertentu (variabel tunggal). Permasalahan akan muncul ketika program kita harus mengolah sebanyak satu juta data.

Apakah kita harus membuat satu juta variabel? Bukankah hal tersebut sangat sulit dipraktikkan dalam kode program yang kita tulis? Untuk mengatasi hal tersebut, bahasa pemrograman memiliki suatu alat untuk menyimpan himpunan data ke dalam satu nama variabel yang diberikan indeks. Salah satunya disebut sebagai larik atau *array*.

Contoh di dunia nyata yang merepresentasikan larik adalah seperti loker yang diberikan nomor (lihat Gambar 3.15). Ketika kamu menyimpan barang di loker tersebut, kamu akan mengingat nomor loker tersebut. Kamu juga dapat menyimpan barang di loker dengan nomor yang berbeda. Pada analogi tersebut, nomor pada loker adalah indeks yang kita gunakan untuk mengenali tempat kita menyimpan barang kita tadi.



Gambar 3.15 Loker sebagai Ilustrasi Larik dalam Pemrograman

Menggunakan larik pada program mirip dengan membuat variabel tunggal. Perbedaannya adalah kamu perlu memberikan informasi ukuran dari larik yang akan dibuat dan kamu perlu mengaksesnya dengan menggunakan indeks. Ada banyak cara untuk mengimplementasikan konsep larik ke dalam kode program, salah satunya menggunakan *vector*. *Vector* ini merupakan pustaka bahasa pemrograman C++ yang juga dapat digunakan pada bahasa C. *Vector* dapat dibuat untuk menampung himpunan data yang ukurannya dinamis bergantung pada jumlah (atau ukuran) data yang disimpan di dalamnya.

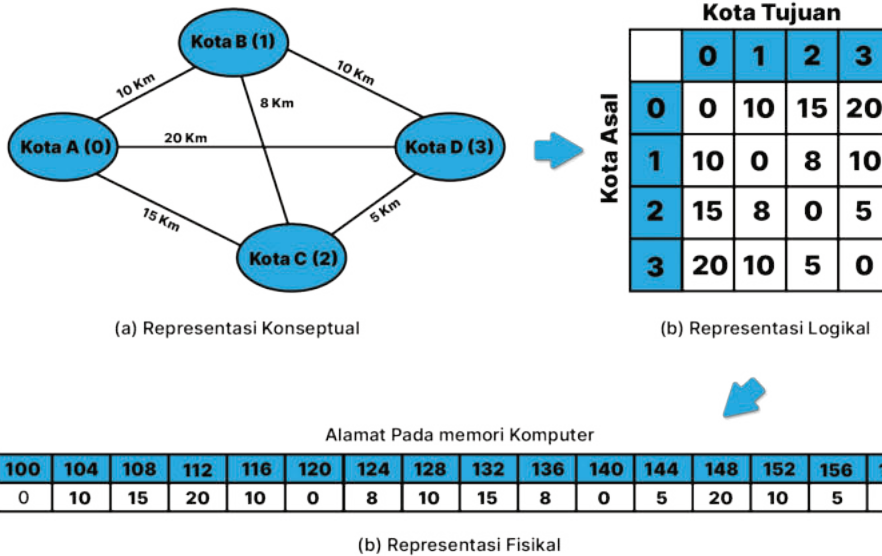
Contoh kode program di bawah ini mengilustrasikan cara mendeklarasikan, menginisialisasi, menyimpan nilai ke, dan mengakses nilai pada suatu *vector*. Berbeda dengan materi pada buku kelas X, program ini ditulis menggunakan bahasa pemrograman C++. Oleh karena itu, kamu perlu menyimpan kode program dengan ekstensi berkas .cpp dan mengkompilasinya dengan kompilator bahasa C++, misalnya GCC.

```
1:  /* Demonstrasi larik satu dimensi dengan vektor */
2:
3:  #include <stdio.h>
4:  #include <vector>
5:  using namespace std;
6:
7:  void cetakLarik(int ukuran, vector<int> larik){
8:      int i;
9:      for(i = 0; i < ukuran; i++)
10:         printf("%d\n", larik[i]);
11: }
12:
13: int main(){
14:     vector<int> bilangan;
15:     int i;
16:     for(i = 0; i < 10; i++)
17:         bilangan.push_back(i);
```

Gambar 3.16 Contoh Kode Larik Satu Dimensi

Sumber: Dela Chaerani/Kemendikbudristek (2024)

Kamu juga dapat membuat larik yang berukuran dua dimensi yang memiliki ukuran berupa baris dan kolom (seperti pada matriks di matematika). Pada kasus seperti apa larik dua dimensi digunakan? Misalnya, kamu ingin menghitung jarak antarkota yang dihubungkan dengan jalan dua arah. Kamu dapat menggunakan matriks dua dimensi untuk merepresentasikan jarak antarkota seperti pada Gambar 3.17 berikut.



Gambar 3.17 Ilustrasi Representasi Panjang Jalan Penghubung Empat Kota (A-D) dalam Bentuk Matriks Dua Dimensi.

Sumber: Dela Chaerani/Kemendikbudristek (2024)

Gambar tersebut memperlihatkan proses berpikir komputasional yaitu abstraksi. Suatu hubungan antarkota di dunia nyata (kontekstual) dibuat dalam bentuk yang lebih sederhana dalam bentuk grafik yang menampilkan informasi yang relevan (konseptual), yaitu hubungan antarkota dan jaraknya. Setelah itu, bentuk tadi diubah ke dalam bentuk yang dapat digunakan dalam menyusun algoritma komputer (logikal) dalam bentuk matriks dua dimensi. Di balik program yang berjalan di komputer kamu, program akan menyimpan matriks tersebut di dalam memori komputer (fisikal) yang memiliki bentuk seperti matriks satu dimensi.

Implementasi larik bisa berbeda-beda di bahasa C atau bahasa lain, yang penting adalah implementasi dari kode program tersebut memenuhi perilaku larik, yaitu adanya himpunan data yang disimpan dan dapat diakses dengan menggunakan suatu indeks. Misalnya, kamu kemudian dapat membuat kode program yang akan menerima isi matriks jarak di atas dan menyimpannya dalam sebuah *vector* dua dimensi. Setelah itu, program akan dapat menjawab jarak antarkota yang ditanyakan oleh pengguna. Perhatikanlah potongan kode program berikut. Setelah itu, perkuat pemahamanmu dengan latihan yang diberikan.

```

1:  /* Program demonstrasi penggunaan larik dua dimensi */
2:
3:  #include <stdio.h>
4:  #include <vector>
5:  #define JUMLAH_KOTA 4
6:  using namespace std;
7:
8:  int main(){
9:      vector<vector<int>> jarak;
10:     int i, j;
11:     int asal, tujuan;
12:
13:     for(i = 0; i < JUMLAH_KOTA; i++){
14:         vector<int> vtr;
15:         for(j = 0; j < JUMLAH_KOTA; j++){
16:             int x;
17:             scanf("%d",&x);
18:             vtr.push_back(x);
19:         }
20:         jarak.push_back(vtr);
21:         vtr.clear();
22:     }
23:
24:     scanf("%d %d", &asal, &tujuan);
25:     printf("Jarak Kota-%d Kota -%d = ", asal, tujuan);
26:     printf("%d km\n", jarak[asal][tujuan]);
27: }

```

Gambar 3.18 Contoh Kode Larik Dua Dimensi

Sumber: Dela Chaerani/Kemendikbudristek (2024)



Ayo Berlatih



Mengidentifikasi dan Menganalisis Contoh Prediksi

Jenis Aktivitas: Kelompok

No Aktivitas: AD-K11-04

Deskripsi Tugas

1. Rancang dan buatlah sebuah program yang membaca N buah bilangan dan mencetaknya secara terbalik. Misalnya, jika diberikan masukan 1 2 3 4 5 6, program akan mencetak 6 5 4 3 2 1.
2. Rancang dan buatlah sebuah program yang membaca sebuah matriks berukuran $N \times M$ dan mencetak hasil *transpose* matriksnya.

3. Modifikasilah kode program untuk menghitung jarak dua kota yang menerima suatu rute. Misalnya, jika rute yang dimasukkan adalah Kota A - Kota B - Kota C, maka nilai total jarak yang dikeluarkan adalah $10 + 8 = 18$ km.
4. Tantangan larik: perhatikan bahwa pada Gambar 3.17 terdapat data yang berganda. Hal ini dikarenakan matriks tersebut menyimpan informasi jarak antara dua kota (misal A dan B) sebagai jarak dari A-B dan B-A. Dapatkan kamu menemukan representasi yang lebih baik daripada contoh tersebut sehingga tidak ada duplikasi informasi dalam penyimpanan data jarak kota?

Praktik Baik Pemrograman

Saat memprogram, kamu akan menggunakan suatu bahasa pemrograman, pustaka, atau perangkat tertentu. Biasakanlah untuk selalu melihat dokumentasi yang tersedia dari teknologi yang kamu gunakan. Dokumentasi ini disediakan oleh pengembang teknologi untuk memberikan informasi tentang teknologi yang mereka sediakan. Pada bahasa pemrograman misalnya kamu dapat melihat penjelasan sintaks dan fitur-fitur yang berkaitan dengan bahasa pemrograman yang kamu gunakan. Ada banyak fitur menarik yang dapat kamu eksplorasi dan manfaatkan di program yang kamu buat.

2. Karakter dan *String*

Selain angka, masukan dari program dapat berupa karakter atau rangkaian karakter seperti kata. Hal ini telah lazim kamu temui, misalnya ketika mengetikkan kata kunci untuk melakukan pencarian di mesin pencari atau ketika kamu memasukkan kata sandi saat *login*. Oleh karena itu, program dilengkapi dengan kemampuan untuk membaca, menyimpan, mengolah, dan mencetak rangkaian karakter tersebut. Rangkaian karakter tersebut dalam pemrograman disebut sebagai *string*.

Definisi *string* dalam pemrograman adalah rangkaian karakter. Karakter sendiri merupakan suatu data berupa huruf, angka, simbol, dan karakter lain yang mengikuti suatu standar tertentu seperti “*American Standard Code for Information Interchange*” (ASCII) atau Unicode. Karakter pada pemrograman

pada umumnya diimplementasikan dalam program menggunakan tipe data *char*. Di sisi lain, ada beberapa cara yang lazim digunakan untuk mengimplementasikan *string* dalam pemrograman.

Pada bahasa C, *string* diimplementasikan sebagai larik karakter yang diakhiri oleh karakter `'\0'`. Dengan kata lain, kamu dapat membuat dan memproses suatu *string* seperti halnya kamu mengolah larik. Bahasa pemrograman lain seperti C++ atau Java memilih sebuah tipe data *string* sendiri yang menyembunyikan beberapa detail yang berkaitan dengan pengelolaan data *string* yang dilakukan oleh program.

Karena proses pada *string* berbeda dengan proses pada bilangan, bahasa pemrograman telah dilengkapi dengan fungsi-fungsi untuk mengolah karakter dan *string*. Misalnya untuk melakukan konversi dari huruf kapital ke non-kapital, penggabungan *string*, pencarian *substring*, dan berbagai fungsi lainnya. Pada bahasa C, kamu dapat mengakses fungsi-fungsi tersebut pada pustaka `<string.h>`. Pada contoh berikut, diberikan dua buah kode program C++ untuk mengolah karakter dan *string*.

Program pertama akan membaca sebuah *string*, mengubahnya ke dalam huruf non-kapital, lalu mencetaknya:

```
1:  /* Demonstrasi karakter dan string */
2:
3:  #include <stdio.h>
4:  #include <iostream>
5:  #include <string.h>
6:  using namespace std;
7:
8:  string lowerCase(string str){
9:      for(int i = 0; i < str.length(); i++)
10:         str[i] = tolower(str[i]);
11:      return str;
12: }
13:
14: int main(){
15:     string str;
16:     cin >> str;
```

Gambar 3.19 Contoh Kode Penggunaan Karakter dan *String* 1

Sumber: Dela Chaerani/Kemendikbudristek (2024)

Program kedua akan membaca banyak *string* hingga membaca *string* “STOP”. Setiap *string* yang dibaca akan diubah ke huruf non-kapital dan dicetak.

```
1:  /* Demonstrasi karakter dan string */
2:
3:  #include <stdio.h>
4:  #include <iostream>
5:  #include <string.h>
6:  using namespace std;
7:
8:  string lowerCase(string str){
9:      for(int i = 0; i < str.length(); i++)
10:         str[i] = tolower(str[i]);
11:     return str;
12: }
13:
14: int main(){
15:     string str;
16:     cin >> str;
17:     while(str != "STOP"){
18:         cout << lowerCase(str) << '\n';
19:         cin >> str;
20:     }
21: }
```

Gambar 3.20 Contoh Kode Penggunaan Karakter dan *String* 2

Sumber: Dela Chaerani/Kemendikbudristek (2024)

Telusurilah kedua program tersebut untuk memahami seperti apa mereka bekerja. Kemudian, salin dan jalankan kedua program tersebut menggunakan kompilator bahasa C++. Jika diperlukan, pelajailah dokumentasi pustaka *string.h* di C dan tipe data *string* pada C++.



Ayo Berlatih

Latihan Karakter dan String

Jenis Aktivitas: Individu

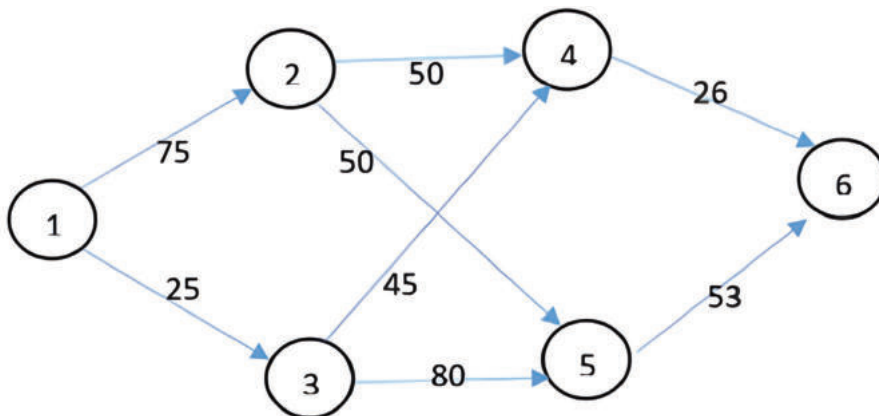
No Aktivitas: SAP-K11-09

Deskripsi Tugas

1. Rancang dan buatlah sebuah program yang dapat membaca sebuah string dan mencetaknya secara terbalik.
2. Rancang dan buatlah sebuah program yang membaca sebuah kata sandi dan mencetak jumlah karakter yang berupa angka, huruf, dan simbol. Bedakan huruf kapital dan non-kapital.

D. Uji Kompetensi

Seorang sopir ambulans harus mencari rute terpendek untuk menjemput pasien yang kritis. Adapun rute yang tersedia adalah sebagai berikut.



Gambar 3.21 Jarak dan Alternatif Rute Ambulans

Sumber: Dela Chaerani/Kemendikbudristek (2024)

Jika Ambulan berada di titik 1 sedangkan pasien berada di titik 6 dan kondisi jalan dalam keadaan tanpa hambatan. Carilah lintasan terpendek dengan melakukan perhitungan setiap lintasan yang memungkinkan. Selanjutnya, bandingkanlah setiap lintasan tersebut untuk mencari nilai terkecil agar menjadi lintasan terpendek yang kita cari. Silakan gunakan strategi pemrograman dinamis untuk memecahkan permasalahan ini!